

SymPy Bug Report

1 Bug 14: Rayleigh CDF is positive below support

1.1 Minimal Reproducer

```
from sympy.stats import P, Rayleigh, cdf

R = Rayleigh("R", 1)

print("cdf(R)(-1):", cdf(R)(-1))
print("P(R <= -1):", P(R <= -1))
print("expected cdf below support:", 0)
print("incorrect:", cdf(R)(-1) != 0)
```

1.2 Observed Behavior

```
cdf(R)(-1): 1 - exp(-1/2)
P(R <= -1): 0
expected cdf below support: 0
incorrect: True
```

The direct CDF call returns the positive expression $1 - \exp(-1/2)$ at a negative argument. This is inconsistent with the same distribution queried through $P(R \leq -1)$, which correctly returns 0.

1.3 Expected Behavior

For a Rayleigh random variable R with scale parameter $\sigma > 0$, $\text{cdf}(R)(z)$ should be 0 for every $z < 0$. In the representative case $R = \text{Rayleigh}(1)$, the mathematically correct result is therefore

$$\text{cdf}(R)(-1) = 0.$$

1.4 Mathematical Explanation

The Rayleigh distribution has support $[0, \infty)$. Its density is

$$f_{\sigma}(x) = \begin{cases} \frac{x}{\sigma^2} \exp\left(-\frac{x^2}{2\sigma^2}\right), & x \geq 0, \\ 0, & x < 0, \end{cases}$$

so its CDF is the corresponding piecewise function

$$F_{\sigma}(z) = \begin{cases} 0, & z < 0, \\ 1 - \exp\left(-\frac{z^2}{2\sigma^2}\right), & z \geq 0. \end{cases}$$

The closed form $1 - \exp(-z^2/(2\sigma^2))$ is only the nonnegative branch of the CDF. Extending that branch to negative z is not an equivalent representation: for $z < 0$, the event $R \leq z$ is empty because no Rayleigh-distributed value can be negative. Thus the returned value is not a harmless simplification, branch choice, or alternate form; it assigns positive probability to a point below the distribution support.

1.5 Source-Level Diagnosis

The diagnosis identifies a support-guarding gap in the Rayleigh distribution’s custom CDF implementation. In `sympy/stats/crv_types.py`, `RayleighDistribution` declares the nonnegative support `Interval(0, ∞)`, but its `_cdf` method returns the unguarded nonnegative-branch formula $1 - \exp(-x^2/(2\sigma^2))$. The traced call path then reaches `SingleContinuousDistribution.cdf` in `sympy/stats/crv.py`. That method calls a distribution-specific `_cdf` first and returns it directly whenever it is not `None`, so the generic `compute_cdf` path that would account for support is bypassed.

```
class RayleighDistribution(SingleContinuousDistribution):
    set = Interval(0, oo)

    def _cdf(self, x):
        sigma = self.sigma
        return 1 - exp(-(x**2/(2*sigma**2)))

def cdf(self, x, **kwargs):
    if len(kwargs) == 0:
        cdf = self._cdf(x)
        if cdf is not None:
            return cdf
    return self.compute_cdf(**kwargs)(x)
```

This source-level behavior explains the observed failure: for $x = -1$, the Rayleigh-specific `_cdf` returns the algebraic expression $1 - \exp(-1/2)$, and the caller accepts it before any support condition can replace it by 0. A plausible fix direction, supported by the diagnosis, is to add the missing support guard to the Rayleigh CDF, for example by returning a piecewise expression whose below-support branch is 0, or otherwise ensuring that custom CDFs preserve the declared support.

1.6 Additional Failing Instances

The independent verification covers the representative case and five additional instantiations with positive scale parameters and negative evaluation points. In every row, $z < 0$, so $R \leq z$ is an empty event for a Rayleigh random variable. The expected result is therefore 0 across the family, regardless of the scale value.

Input / Expression	SymPy behavior	Expected behavior
$\sigma = 1, z = -1: \text{cdf}(R)(z)$	$1 - \exp(-1/2)$, while $P(R \leq z) = 0$	0
$\sigma = 2, z = -1: \text{cdf}(R)(z)$	$1 - \exp(-1/8)$, while $P(R \leq z) = 0$	0
$\sigma = 1, z = -2: \text{cdf}(R)(z)$	$1 - \exp(-2)$, while $P(R \leq z) = 0$	0
$\sigma = 3, z = -5: \text{cdf}(R)(z)$	$1 - \exp(-25/18)$, while $P(R \leq z) = 0$	0
$\sigma = 1/2, z = -1: \text{cdf}(R)(z)$	$1 - \exp(-2)$, while $P(R \leq z) = 0$	0
$\sigma = \sqrt{2}, z = -3: \text{cdf}(R)(z)$	$1 - \exp(-9/4)$, while $P(R \leq z) = 0$	0

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and found a **novel** verdict, which should be framed as a new failure mode with no public precedent. The only related public material identified in the artifact is a discrete-support issue, SymPy issue #20031, which is related in theme because it concerns support handling, but it is not an exact duplicate of this Rayleigh CDF failure. The present bug remains individually actionable because it gives a concise reproducible case, a concrete affected distribution method, and a targeted regression test for the continuous Rayleigh CDF.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Rational, S, sqrt
from sympy.stats import Rayleigh, cdf

CASES = [
    (S.One, S.NegativeOne),
    (S(2), S.NegativeOne),
    (S.One, S(-2)),
    (S(3), S(-5)),
    (Rational(1, 2), S.NegativeOne),
    (sqrt(2), S(-3)),
]

@pytest.mark.parametrize("sigma,z", CASES)
def test_rayleigh_cdf_is_zero_below_support(sigma, z):
    R = Rayleigh("R", sigma)
    assert cdf(R)(z) == 0
```